

Avaliação Técnica de uma Arquitetura Web baseada em BaaS/Serverless para Intermediação de Serviços

Um estudo de caso da plataforma [Hubservi](#).

Segurança

Desempenho

Testabilidade

Confiabilidade

[Autor 1]
Apresentador A

[Autor 2]
Apresentador B

[Orientador(a)]
Orientação

[Instituição]
2026

No BaaS, a segurança mora no banco — como validá-la?

ARQUITETURA TRADICIONAL

Um **servidor de aplicação** próprio intermedeia o acesso: nele moram autenticação, autorização e regras de negócio.

BAAS / SERVERLESS (HUBSERVI)

Sem servidor próprio. Autorização e integridade migram para o **banco**: políticas de acesso (RLS), *triggers*, restrições — configuração declarativa.

QUESTÃO DE PESQUISA

Como **avaliar tecnicamente** uma arquitetura BaaS/Serverless por meio de métricas e testes de Engenharia de Software, considerando os atributos de qualidade?

O que entregamos é a avaliação — não o sistema

OBJETO DE PESQUISA

A avaliação técnica

O método de avaliar a arquitetura — reproduzível e transferível a outras aplicações BaaS. É a contribuição científica.

INSTRUMENTO

O Hubservi — desenvolvido 100% por nós

O laboratório que torna a avaliação possível.

Avaliar RLS e *triggers* exige acesso irrestrito ao código e ao banco → só possível num sistema próprio. **O sistema é o instrumento; a avaliação é o resultado.**

Como construímos o instrumento

PROCESSO – ENTREGA INCREMENTAL POR MARCOS

- M1 Requisitos e arquitetura
- M2 Fluxos centrais — auth, serviços, booking
- M3 Segurança e governança (RLS, triggers)**
- M4 Validação de qualidade
- M5 Evolução da descoberta de serviços

STACK – CLIENTE

React TypeScript Vite React Query Zod
Tailwind

STACK – BAAS (SUPABASE)

Auth PostgreSQL RLS Triggers

Sem servidor de aplicação próprio — é o que caracteriza o caso.

Escopo MVP · requisitos RF-xx / RNF-xx priorizados · **lint + testes + revisão a cada alteração**

Artefatos que sustentam o trabalho

CONCEPÇÃO E PLANEJAMENTO

TAP

EAP

Canvas

SWOT

5W2H

REQUISITOS

RF-xx funcionais

RNF-xx não funcionais

Priorizados e rastreáveis.

MODELAGEM

UML

BPMN

DER + dicionário

Casos de uso, classes, componentes, sequência, implantação.

A modelagem **não é anexo**: atende aos **objetivos específicos 2 e 3** — e é dela que derivamos a **árvore de utilidade do ATAM** e os **cenários de avaliação**.

OS 4 FLUXOS CRÍTICOS QUE SAEM DESSA MODELAGEM – A SUPERFÍCIE QUE OS TESTES EXERCITAM

Autenticação

→

Serviço

→

Contratação

→

Avaliação

Como avaliamos

PARA CADA ATRIBUTO DE QUALIDADE (ISO/IEC 25010)

Atributo → Cenário → Métrica → Critério → Ferramenta

A DISCIPLINA

Medir → Detectar → Corrigir → Re-medir

Cada estado com evidência guardada. **Nenhum número entra no trabalho sem um arquivo de evidência** — regra anti-fabricação.

O sistema não se reconstruía sozinho

AMBIENTE LIMPO

permission denied for table profiles

Reconstruir o banco **só com o histórico versionado** falhou: as permissões de acesso à API nunca estiveram no controle de versão — em produção, foram aplicadas à mão.

Corrigido com uma *migration* que torna o histórico auto-suficiente.

Fragilidade típica do BaaS: parte da configuração vive no serviço gerenciado e escapa ao versionamento. **Só uma avaliação reprodutível expõe isso.**

Dois furos reais de segurança

FURO 1 · DADOS PESSOAIS

PII exposta a qualquer autenticado

ANTES

lê e-mail/telefone de todos



DEPOIS

só o próprio registro

- Após correção: 0 acessos indevidos nos cenários testados

FURO 2 · AVALIAÇÃO

Review atribuída ao prestador errado

ANTES

aceita provider_id falso



DEPOIS

trigger valida o dono

"Não construímos testes para confirmar que o sistema estava bom – construímos para **descobrir onde estava ruim**. O valor está em ter, primeiro, a tela vermelha."

Confiabilidade

● atende

- ✓ Máquina de estados recusa transições inválidas
- ✓ Integridade referencial: 0 registros órfãos
- ✓ Fluxo ponta a ponta (login → avaliar) completa

Testabilidade

evoluiu

18% → 32% cobertura

O essencial não é a média: os **módulos que controlam acesso** estão em **82–100%**, com trava automática que impede regressão.

Desempenho — os dois lados

API SOB CARGA

253 ms

● p95 ✓

PÁGINA (ABRIR)

3,2 s

● LCP ✗

O contraste **localiza o gargalo no frontend** (peso do *bundle*), não no backend.

Manutenibilidade — diagnóstico

✓ **0 dependências circulares** (2 ferramentas)

✗ Duplicação nos 2 *dashboards* : **61 linhas** repetidas

Estrutura sólida; higiene de código abaixo do ideal, com alvo nomeado.

A contribuição é o método

Segurança

atende*

Confiabilidade

atende

Testabilidade

evoluiu

Desempenho

backend ✓ / frontend ✗

Manutenibilidade

estrutura ✓ / higiene ✗

* após corrigir 2 furos; 1 CVE de produção a tratar; DAST pendente.

Um **procedimento reprodutível de avaliação**, transferível a outras aplicações BaaS/Serverless.

Não atestamos que o sistema é uniformemente bom — e é a capacidade de **localizar deficiências com precisão** que evidencia o valor do método.

Medimos, detectamos, corrigimos — e dissemos a verdade sobre o sistema.

18→32%

cobertura

253 ms

API p95

3,2 s

LCP (frontend)

2

furos corrigidos

0

ciclos

[Autor 1] · [Autor 2]

Hubservi · Avaliação técnica

github.com/Richardy-Rodrigues/hubservi

Evidências em docs/tcc/medicoes/